

배고픈 개발자들의 “먹자GO” 서비스 발표

먹자GO : Spring Boot 배달 서비스 백엔드 프로젝트

본 발표는 Spring Boot 기반 배달 서비스 백엔드 프로젝트의 진행 과정을 되돌아보고, 구현 내용과 기술적 의사결정, 협업 과정에서의 배움을 공유하기 위해 구성되었습니다.

배고픈 개발자들 | 2026.07

이규민 박수빈 김태희 황지호 최필규 정승호



발표 목차

01 프로젝트 소개 요약

02 시스템 아키텍처

03 ERD 설계

04 주요 동작 소개

05 핵심 구현 내용

06 문제 해결

07 협업 방식

08 협업 중 발생한 문제 & 해결방안

09 프로젝트 회고

프로젝트 소개 요약

먹자GO / 기간: [2026.07.02 ~ 2026.07.20]

고객·사장·관리자 역할 기반으로, 주문 → 결제 → 리뷰까지 이어지는 배달 주문 플랫폼. 메뉴 설명은 AI로 자동 생성.

✓ 핵심 도메인

- **인증/회원**(User·Auth·Address)
- **가게·메뉴**(Store·Menu, 메뉴 AI 설명 생성)
- **주문·결제·리뷰**
(Order·Payment·Review)
- **분류·지역**(Category·Region)
- **이미지**(Image·S3)

✓ 기술 스택

- **Backend**: Spring Boot 3, Java 17
- **인증·인가**: Spring Security, JWT 기반 Stateless 인증 및 역할별 접근 권한 제어
- **영속성**: Spring Data JPA, QueryDSL을 활용한 CRUD 및 동적 조회
- **Database**: PostgreSQL 운영 DB, H2 테스트 DB, AWS RDS
- **AI**: Spring AI, Gemini, RAG 기반 메뉴 설명 생성
- **파일 저장소**: AWS S3, Presigned URL 기반 이미지 업로드
- **배포·운영**: Nginx, Docker, Docker Hub, GitHub Actions 기반 CI/CD
- **공통 설계**: Soft Delete, 공통 응답 포맷(ApiResponse, PageResponse) 적용

팀원 역할별 소개



주문·리뷰



공통·인프라



메뉴·이미지



가게·분류·지역



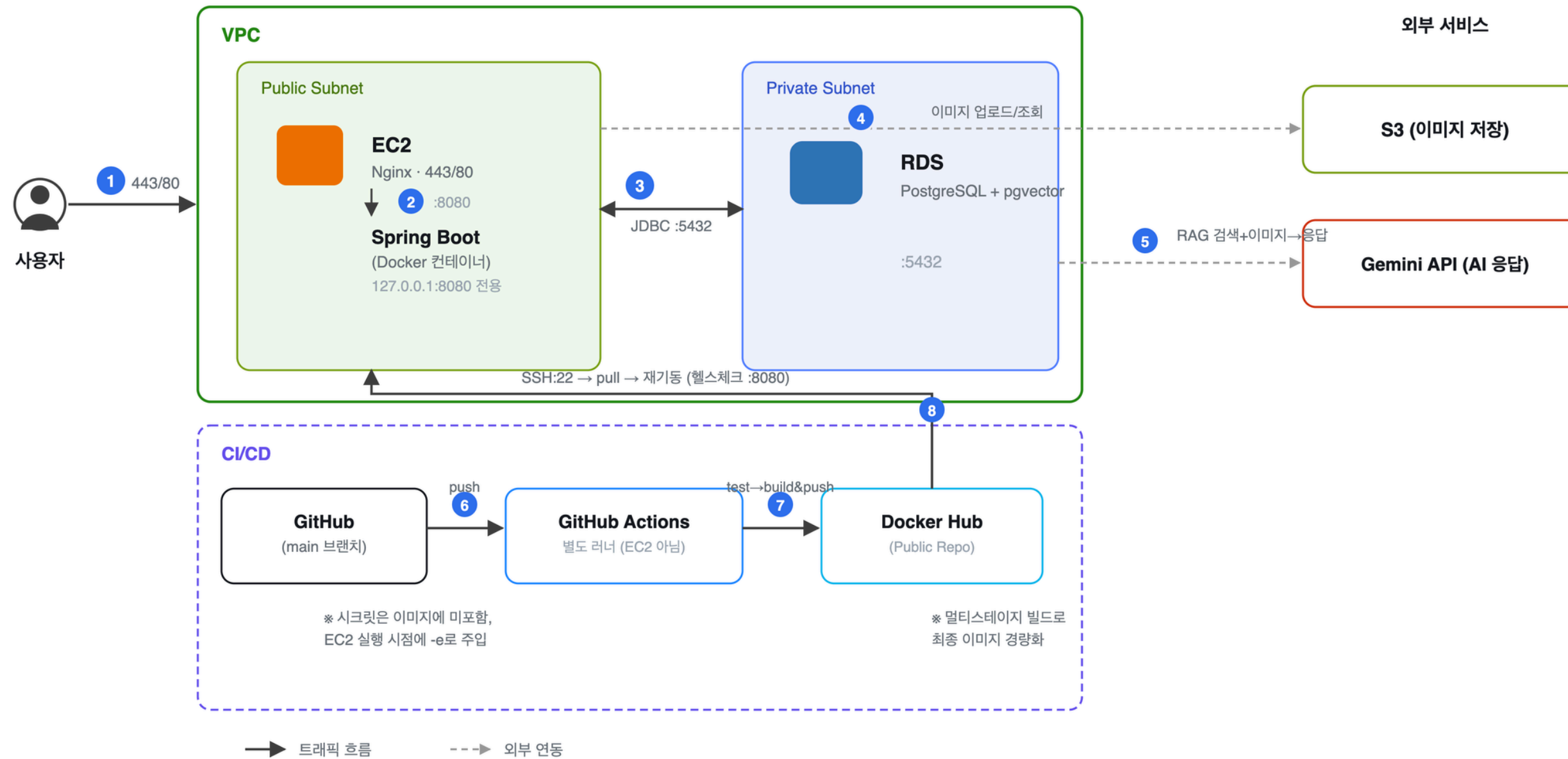
결제



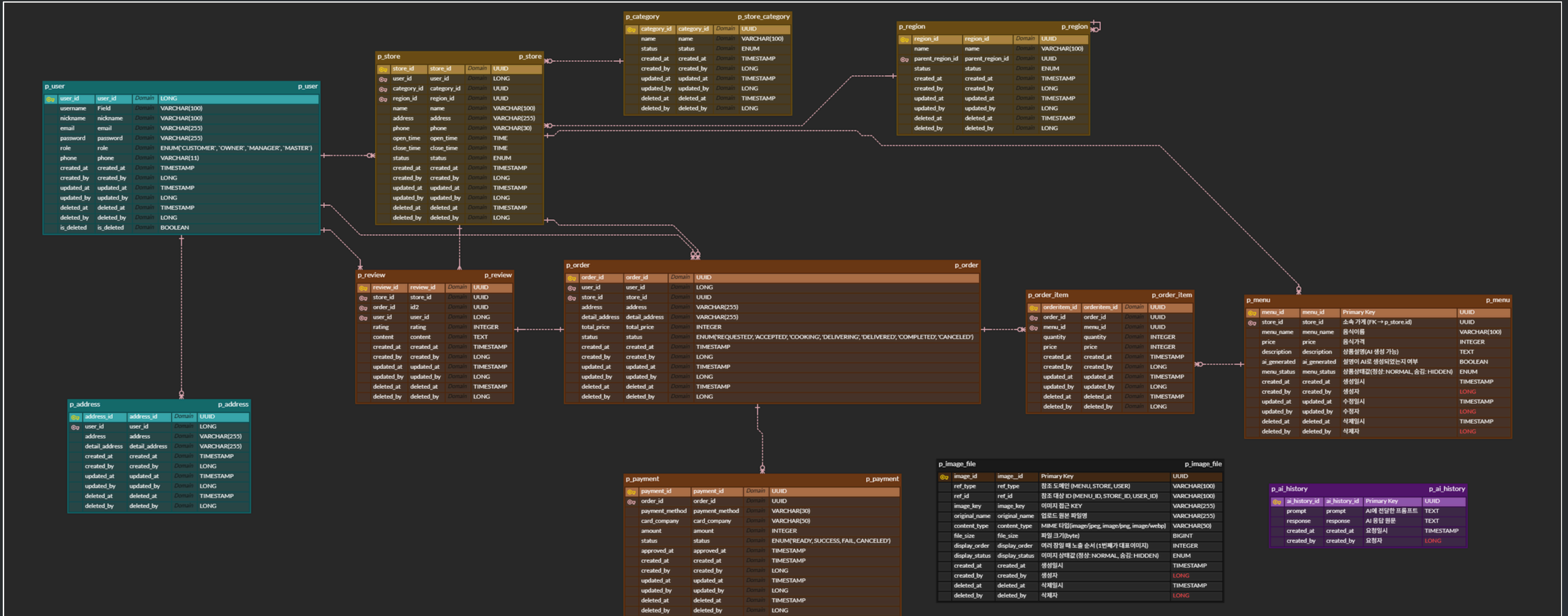
인증/회원

시스템 아키텍처

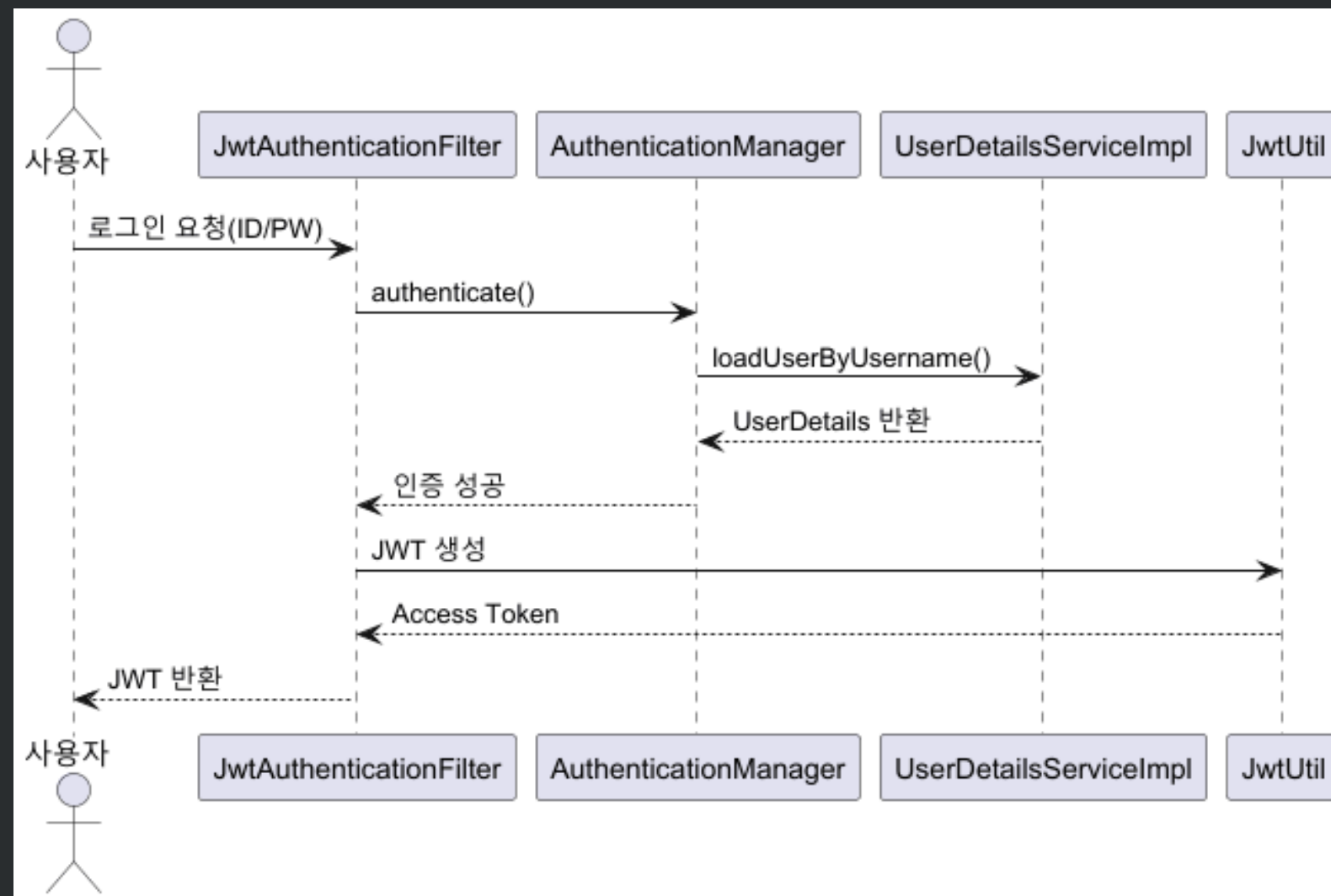
배달 주문 관리 플랫폼 - 백엔드 아키텍처 (Nginx + Docker Hub 기반 배포)



ERD 설계



주요 동작 흐름



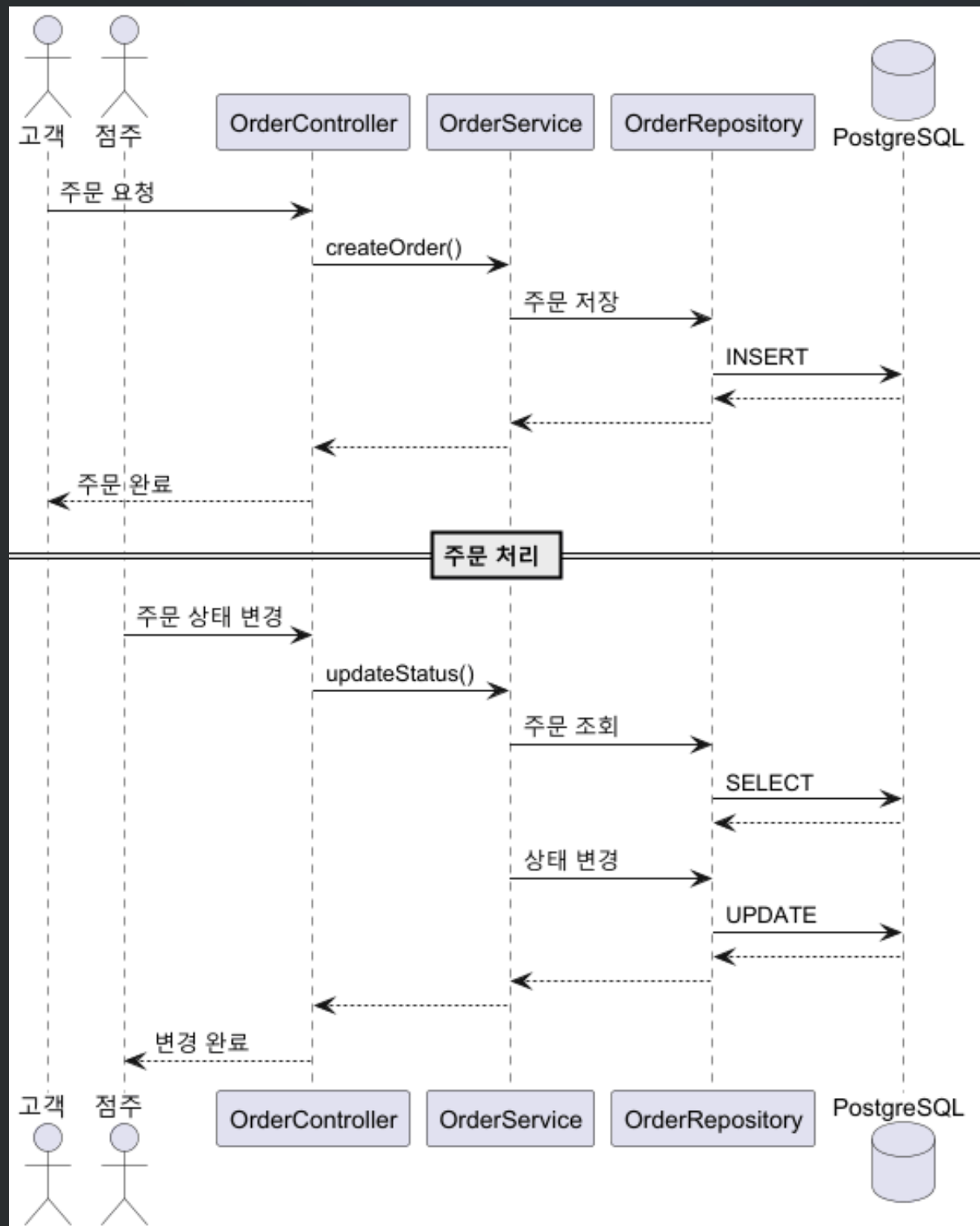
✓ 구현 기능

- jwt방식을 기반의 인증 방식, 세션을 이용하여 로그인 상태를 저장하지 않고, 토큰을 검증하도록 구현
- user 테이블의 is_deleted 도입
 - 만약 사용자 복구 시 해당 값만 false로 바꾸어주면 바로 복구할 수 있기 때문에 확장성을 고려

✓ 어려웠던 점 & 해결 방안

- 유저 테이블의 nickname과 email, username이 유니크 키로 설정되어 삭제된 유저임에도 해당 이메일이나 닉네임을 사용할 수 없었음
 - → 기존 유니크 키를 지우고 부분 유니크 인덱스를 사용하여 해결

주요 동작 흐름



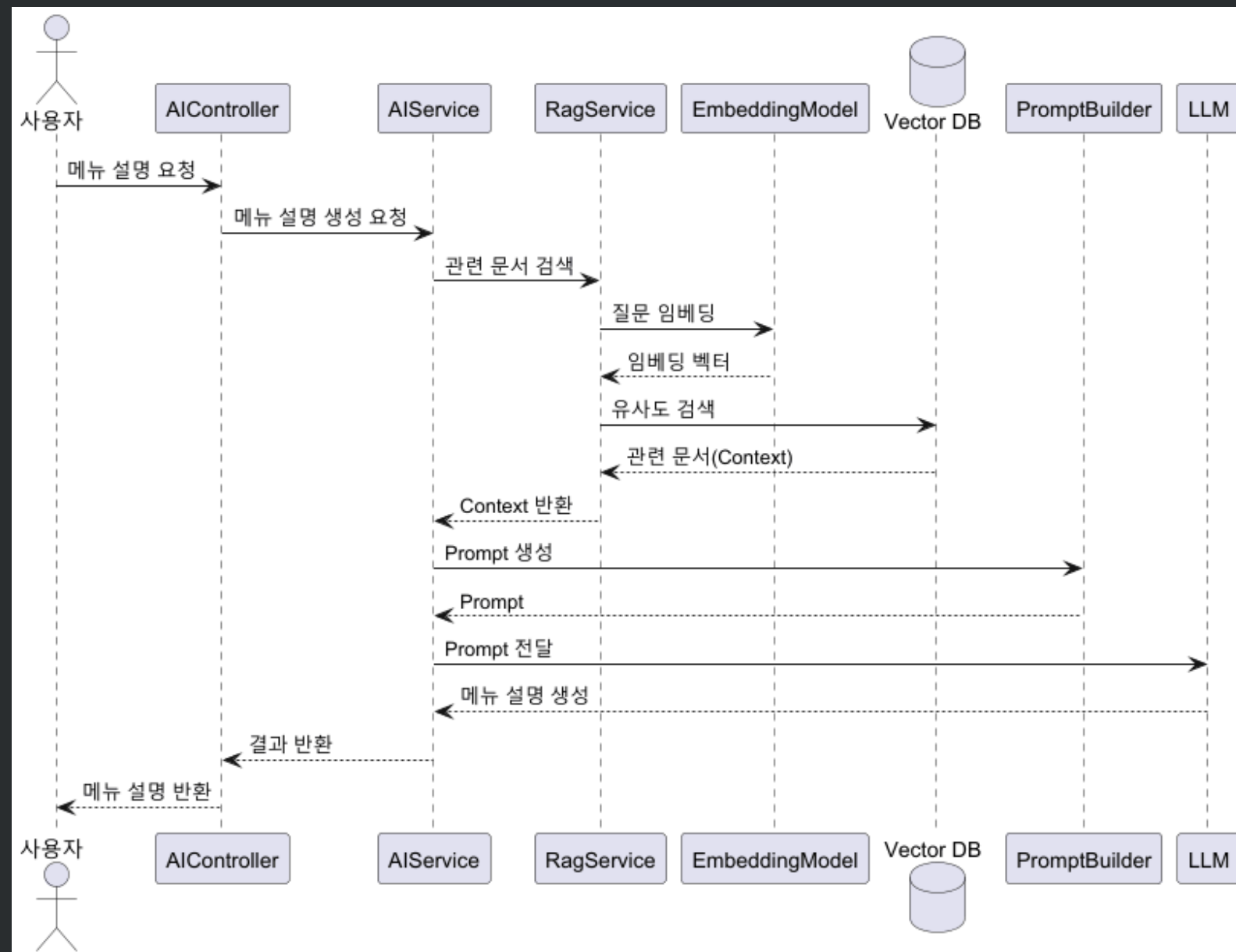
구현 기능

- 상태 전이 — 'REQUESTED→ACCEPTED→...→DELIVERED' 는 enum이 허용한 전이만 가능하도록 함. (임의 값 차단), 취소는 별도 규칙
- 조회 — 역할별 범위(고객=본인 / 사장=본인 가게 / 관리자=전체)
- 취소/삭제 — 취소는 본인·5분 이내, 삭제는 soft delete
- 부분 유니크 인덱스 + saveAndFlush

어려웠던 점 & 해결 방안

- N+1 문제 — 데이터 모델별 최적화
 - 'findAllById' + 'Map' 배치
- Soft Delete × Unique × 동시성 (리뷰) — 연쇄 트러블슈팅
 - 부분 유니크 인덱스('WHERE deleted_at IS NULL') → 재작성 허용 + 동시 중복 방지 동시 달성
 - save 대신 saveAndFlush + catch(DataIntegrityViolationException) 로 제약 위반을 즉시 잡아 500 → 409 변환

주요 동작 흐름



✓ 구현 기능

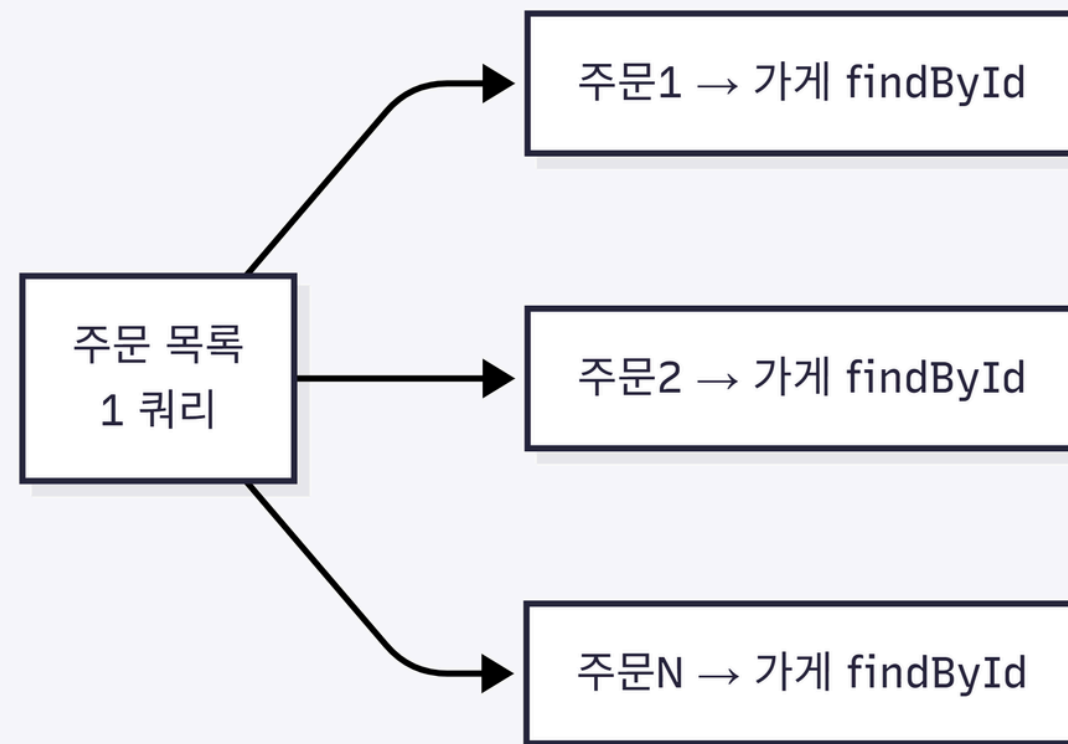
- LLM/RAG 기능
 - 사용자가 입력한 메뉴명, 카테고리, 요청 내용을 바탕으로 Vector DB에서 관련 음식 문서를 검색
 - 해당 문맥을 프롬프트에 포함해 AI 메뉴 설명을 생성
- 단순한 AI 답변이 아니라 카테고리별 특징이 반영된 자연스러운 설명을 제공
- Spring AI를 사용해 추후 다른 LLM 모델로도 쉽게 교체할 수 있도록 구현

✓ 어려웠던 점 & 해결 방안

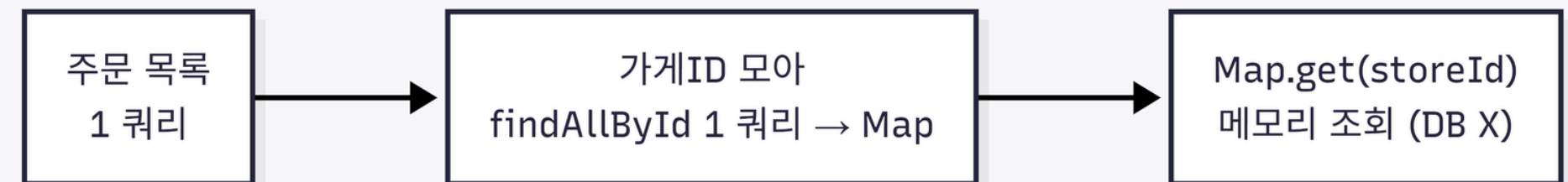
- LLM 호출 자체보다 RAG 흐름을 구성하는 과정
 - 카테고리별 문서를 작성하고, 문서를 임베딩해 Vector DB에 저장한 뒤, 사용자 요청과 유사한 문서 조각을 검색해 프롬프트에 결합하는 방식으로 해결

N + 1 문제 해결 (findAllById + Map)

❌ Before : 1 + N

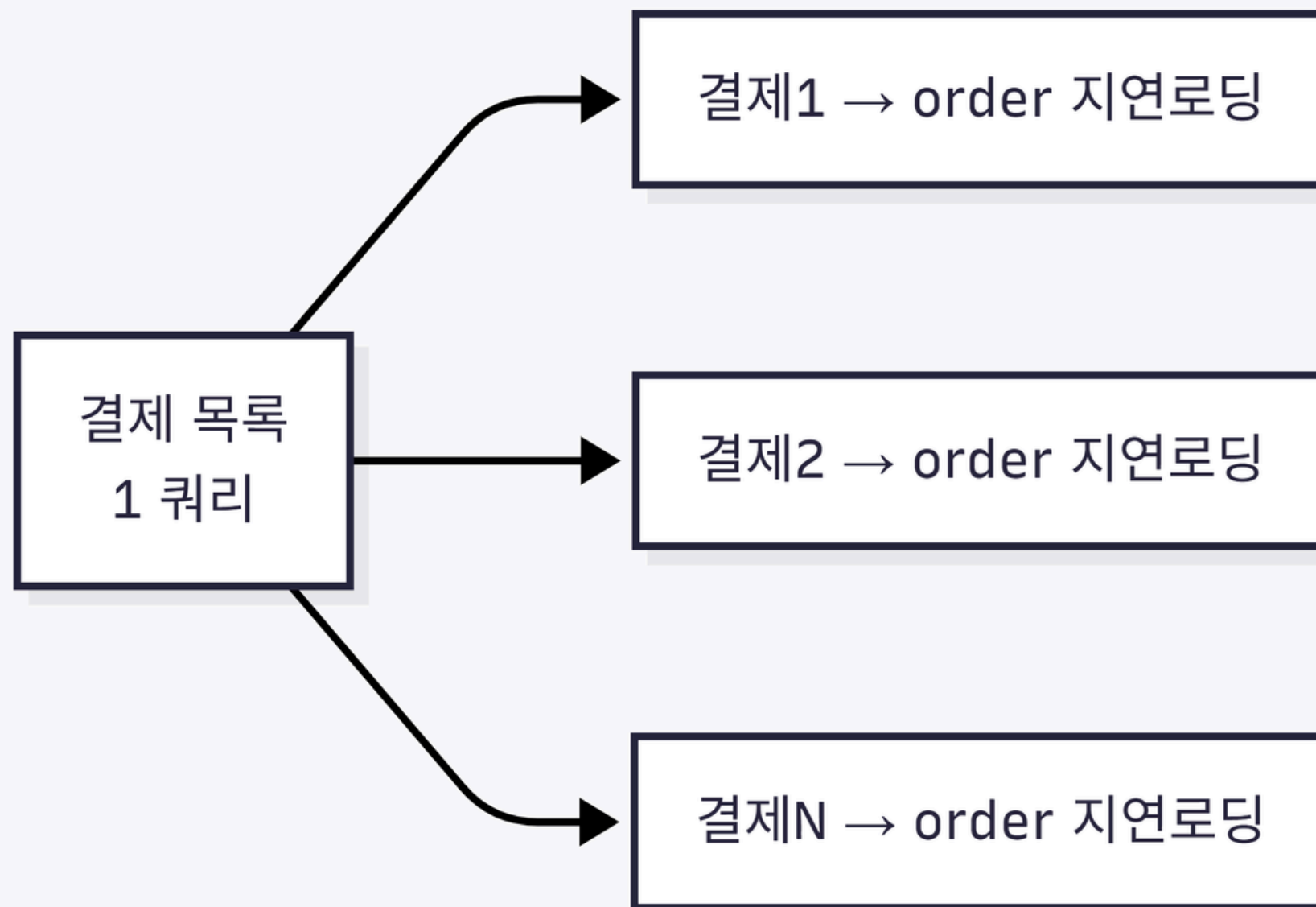


✅ After : 2 (목록 + 배치)



N + 1 문제 해결 (@EntityGraph / fetch join)

❌ Before : 1 + N

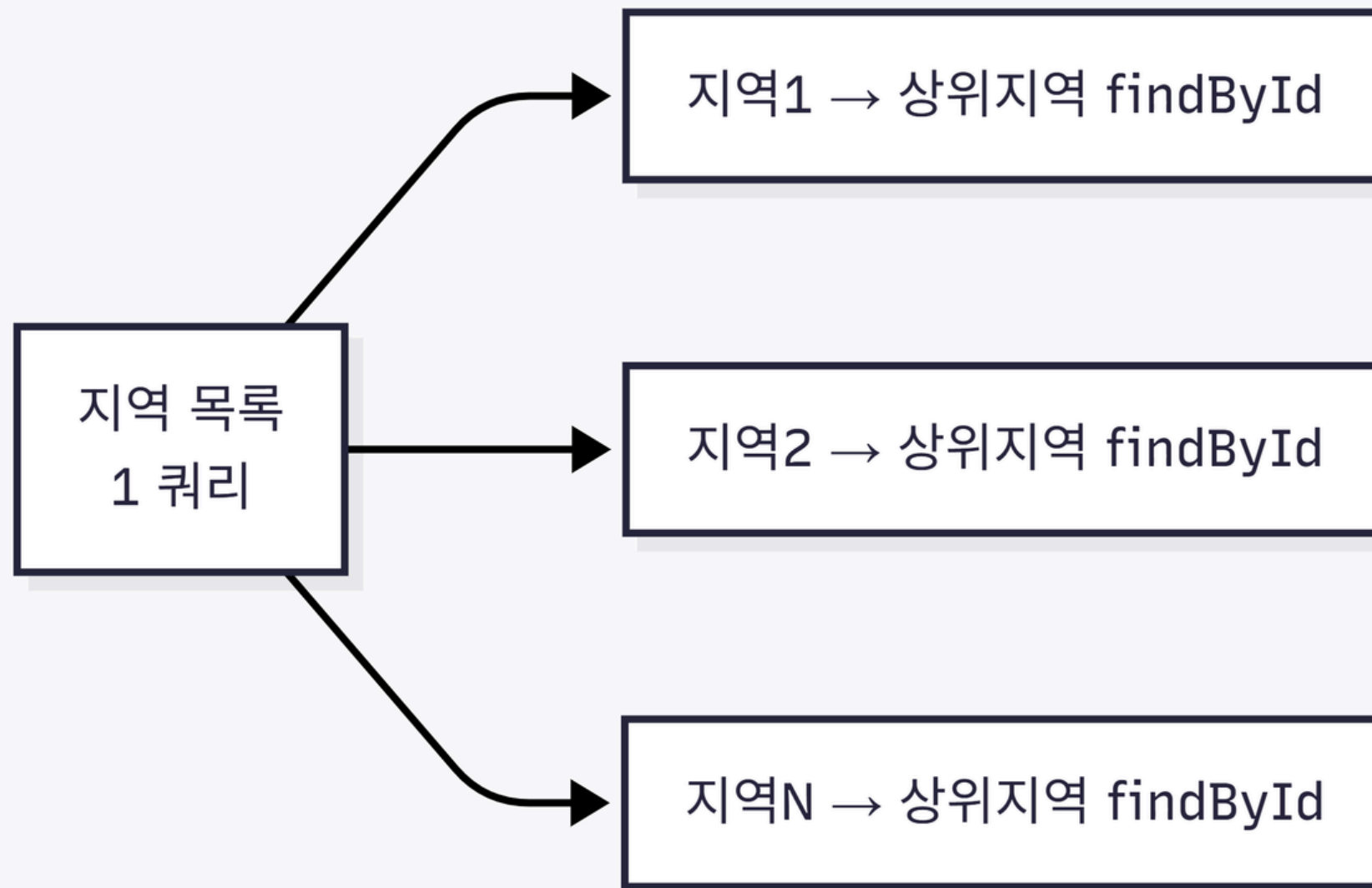


✅ After : 1 쿼리

결제 JOIN FETCH order
(연관 매핑 활용)

N + 1 문제 해결 (QueryDSL self-join)

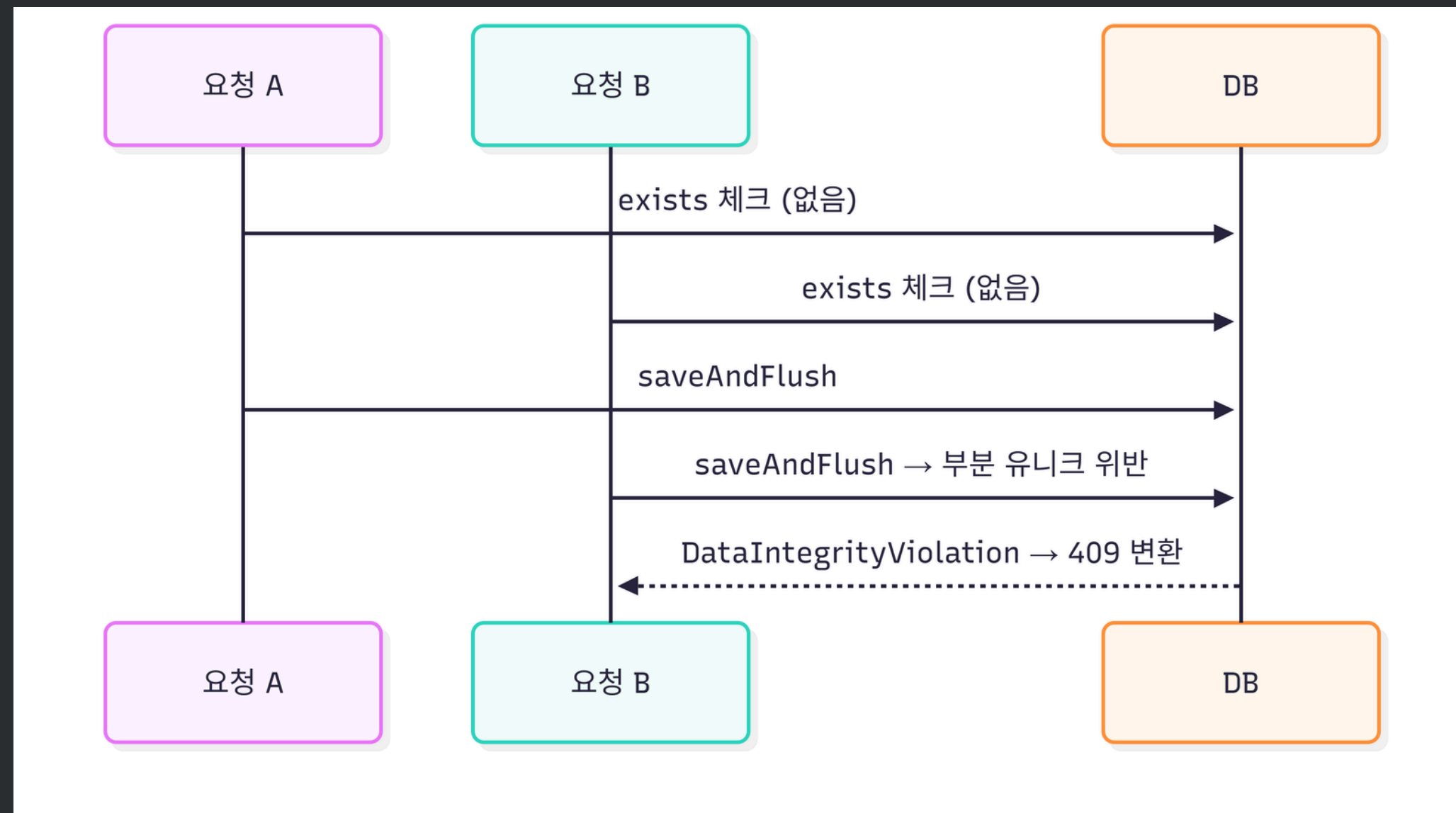
✗ Before : 1 + N



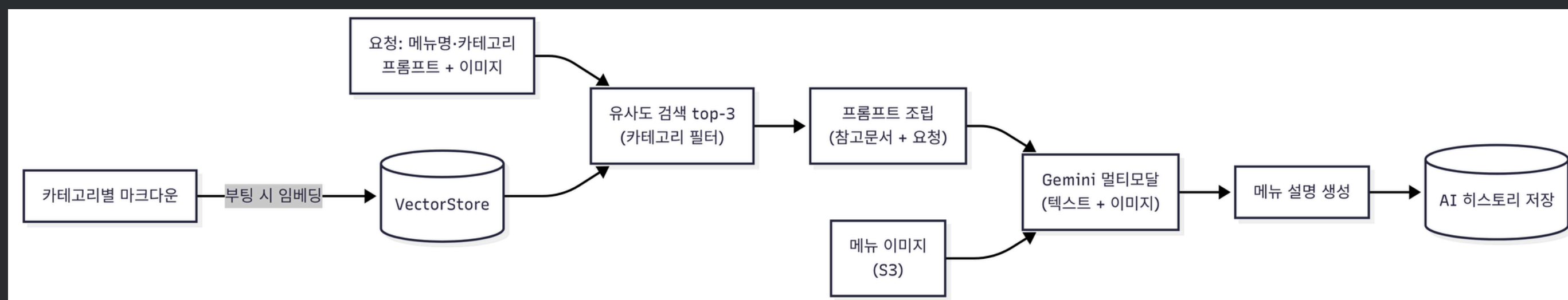
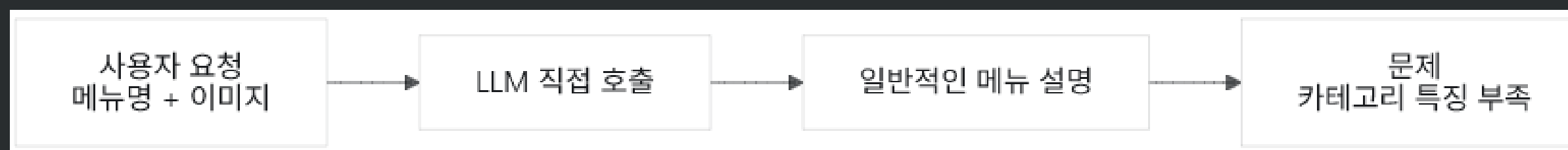
✓ After : 1 쿼리

지역 ⋈ 상위지역
self-join ON
(parent_id = region_id)

Unique 제약 & 동시성 — 연쇄 트러블슈팅



LLM 답변 문제 해결 (RAG 도입)



협업 방식

☒ **Require a pull request before merging**
 Require all commits be made to a non-target branch and submitted via a pull request.

☐ **Require status checks to pass**
 Choose which status checks must pass before the ref is updated. When all checks pass, the ref is updated.

☒ **Block force pushes**
 Prevent users with push access from force pushing to refs.

- fix/#45/duplicate-Config
- fix/#39/user
- feat/#35/store-API
- feat/#31/address

Git 브랜치 전략

모든 기능 단위 작업은 Issue를 생성 후 진행.
 타입/#이슈번호/기능명으로 브랜치 생성.
 직접 push/force-push/브랜치 삭제 막음.

```

98 + //지역 삭제
99 + @Transactional
100 + public ResDeleteRegionDto deleteRegion(UUID regionId) {
101 +
  
```

subeeen 2 days ago

삭제할 때 이 지역을 부모로 삼고 있는 하위 지역이 있는지 확인하는 로직이 없는데 하위 지역이 있으면 삭제를 막을지, 같이 삭제(cascade)할지 정해서 진행해주시면 좋을 것 같습니다. 저는 삭제를 막는게 나을 것 같습니다!

jiho0107 yesterday

거기까진 미처 고려하지 못했네요!
 삭제를 막는 방향으로 수정하겠습니다!! 감사합니다

```

80 + public ResponseEntity<ApiResponse<PageResponse<ResOrderListDto>>> getOrders(
81 +     @AuthenticationPrincipal UserDetailsImpl userDetails,
82 +     @RequestParam(defaultValue = "ALL") String status,
  
```

jiho0107 3 days ago · edited

status 는 String 으로 받기 보다는 Enum 타입으로 클래스를 만들어서 OrderStatus 와 같은 타입으로 받는 게 더 안전하지 않을까 싶습니다. String 으로 받으면 service 레벨에서 다시 검증해야 하지만 Enum으로 받으면 스프링이 자동 검증을 하는 것으로 알고 있습니다.
 그리고 프로젝트 전반적으로도 좀 더 일관성이 생길 것 같습니다.

aa04260 3 days ago

주문 목록 조회 시에 필터링을 하기 위해서 enum에 없는 "ALL" 값을 받아야했기 때문에 사용했는데, 일관성은 문제가 되네요 수정해보겠습니다~

코드 리뷰 문화

인프라 담당 : 박수빈 (@subeeen)
 자동 리뷰어 지정.
 머지 전 무조건 코드리뷰 수행.

공유

메시지 🎉💖🥰 📎 파일 및 링크 + 🔍 이 채널에서 메시지 검색(⌘F)

이규민 오후 12:39

명세서랑 통일했습니다

postgresql_schema 수정.sql ▼

```

1 -- PostgreSQL DDL
2
3 CREATE TYPE store_status AS ENUM ('OPEN', 'CLOSED', 'BREAK_TIME', 'INACTIVE');
4 CREATE TYPE menu_status AS ENUM ('NORMAL', 'HIDDEN');
5 CREATE TYPE category_status AS ENUM ('ACTIVE', 'INACTIVE');
  
```

7월 7일 화요일

이규민 오후 1:58

주문 도메인 서비스 사전 리뷰 요청 - feat/#7/order

주문 서비스 7종(생성/목록/단건/상품목록/상태변경/취소/삭제) 먼저 리뷰받고 싶습니다
 로직·구조 위주로 봐주세요. 컨트롤러 연결·JWT·QueryDSL은 아직입니다.

menu/store 도메인 병합 전이라 브랜치 단독 빌드는 안 됩니다(정상). 상세는 스레드에 남겨놓겠습니다

4개의 댓글 6월 전 마지막 댓글

7월 8일 수요일

커뮤니케이션 방식

슬랙 채널 개설을 통해 편리하게 소통.
 노선을 활용해 자료 정리/공유.

협업 중 발생한 문제 & 해결 방안



✓ 공통 파일 충돌 (application.yml / ErrorCode)

- 문제: 여러 도메인이 application.yml(인증 토큰·API 설정)과 ErrorCode를 동시에 수정 → 머지 충돌 빈발.
- 해결: 공통 파일 변경은 PR에 명시 + 리뷰에서 우선 확인
 - ErrorCode는 도메인 접두어 규칙(ORDER_, REVIEW_)으로 구획해 충돌 최소화



✓ 인증 미완성 상태의 병렬 개발

- 문제: 인증 모듈 완성 전에 각 도메인이 병렬로 개발 → 권한 검증을 임시로 주석 처리한 코드 존재.
- 해결: @PreAuthorize + 서비스 검증으로 각 도메인이 방어선을 먼저 심고, 인증 통합 시 연결.
 - 미완 부분은 TODO로 명시해 누락 방지



✓ 스키마 변경 공유 (수동 DDL)

- 문제: 부분 유니크 인덱스처럼 ddl-auto가 자동 생성 못 하는 변경은 팀원 로컬/운영 DB에 각자 반영 필요
- 해결: schema.sql로 도입해 앱 기동 시점에 자동으로 적용. (향후 마이그레이션 도구 도입 과제로 인식)

팀 프로젝트 회고 총평

“왜”를 말로 설명하는 힘: 리뷰를 주고받으며 내 선택의 근거를 언어화하니, 지식이 훨씬 단단해졌다.

개발 방향과 구조를 사전에 맞추면서 merge 과정에서 발생할 수 있는 충돌을 줄이고 안정적인 협업이 가능했음

혼자 해결하기보다 팀원들과 논의하고 합의하는 과정이 더 나은 결과를 만든다는 것을 체감

Learn

설계 단계가 단순한 준비 과정이 아니라 개발 생산성과 유지보수성을 결정하는 중요한 과정임을 느꼈다.

Good

코드리뷰를 담당하며 다른 도메인의 로직 흐름을 파악할 수 있었고, 동시성 문제나 계층간 참조, 순환 문제 등을 파악하고 이야기 할 수 있어 좋았다.

운영 환경 로그 모니터링을 더 해봤으면 좋았을 것.

Miss

시간 부족으로 인한 테스트 미흡

Next

Blue-Green 또는 Rolling Update를 적용해 무중단 배포

AI 히스토리를 특정 도메인에 종속되지 않도록 설계

경청해 주셔서 감사합니다.

함께한 프로젝트, 다음 도전을 준비하다

함께 고생한 팀원들에게 감사드립니다.

배고픈 개발자들 | 2026.07

이규민 박수빈 김태희 황지호 최필규 정승호

